МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ НАУКИ И ТЕХНОЛОГИЙ ИМЕНИ АКАДЕМИКА М.Ф. РЕШЕТНЕВА»

Институт информатики и телекоммуникаций

Кафедра информационно-управляющих систем

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4

по дисциплине «Тестирование программного обеспечения»

Тема: API-тестирование и интеграция в CI/CD

Преподаватель _____ А.Е. Алехина
подпись, дата — инициалы, фамилия

Обучающийся БПА23-01 23151207 _____ М.В. Давыденко
номер группы, зачетной книжки — подпись, дата — инициалы, фамилия

Красноярск 2026 г.

1 ЦЕЛЬ РАБОТЫ

Научиться тестировать API (REST) как вручную, так и автоматически; спроектировать набор API-тестов; интегрировать автотесты (UI и API) в CI-пайплайн с помощью GitHub Actions.

2 ОПИСАНИЕ API И РУЧНЫЕ ТЕСТЫ

Для тестирования используется учебный стенд Contact List App (`https://thinking-tester-contact-list.herokuapp.com`). API предоставляет методы для управления пользователями и контактами в формате JSON. Авторизация происходит с помощью Bearer-токена, выдаваемого при регистрации или входе.

Перед автоматизацией была составлена коллекция из 10 ручных запросов (с использованием Postman), покрывающая основные маршруты:

– `POST /users` — регистрация нового пользователя.
– `POST /users/login` — вход и получение токена авторизации.
– `GET /users/me` — получение данных профиля.
– `POST /contacts` — создание контакта (требует токен).
– Ошибочные запросы (без токена, дубликат email, неверный пароль).

3 АВТОМАТИЗАЦИЯ API-ТЕСТОВ

Для автоматизации был выбран язык Python и библиотека `requests`, а для запуска — фреймворк `pytest`. Этот выбор обусловлен простотой написания HTTP-запросов и отличной читаемостью кода.

Было реализовано 10 автотестов, покрывающих позитивные, негативные сценарии и проверки безопасности. Результаты прогона представлены в таблице 1.

Таблица 1 – Реализованные API-автотесты

№	Название проверки	Тип	Ожидание
1	Успешная регистрация / создание ресурса	Positive	201 Created
2	Регистрация с невалидным email	Negative	400 Bad Request
3	Дубликат email	Negative	400 Bad Request
4	Успешная авторизация (логин)	Positive	200 OK + token
5	Логин с неверным паролем	Negative	401 Unauthorized
6	Запрос профиля без токена	Security	401 Unauthorized
7	Запрос профиля с невалидным токеном	Security	401 Unauthorized
8	Проверка структуры ответа профиля	Contract	200 OK
9	Метод не поддерживается (`POST /users/me`)	Negative	404 Not Found
10	Создание контакта с авторизацией	Positive	201 Created

4 НАСТРОЙКА CI-ПАЙПЛАЙНА

Для непрерывной интеграции (CI) был выбран GitHub Actions. Пайплайн настроен на автоматический запуск тестов при каждом пуше (push) или создании Pull Request в ветку `main`.

Сценарий (`tests.yml`) включает следующие шаги:

1. Загрузка исходного кода из репозитория.
2. Установка Python 3.12 и зависимостей (`pytest`, `requests`, `playwright`).
3. Запуск написанных API-тестов (`pytest test_api.py`).
4. Запуск UI smoke-тестов из ЛР3 (`pytest test_registration.py`).

5. Публикация результатов в виде артефактов JUnit XML для отчётности.

5 РЕФЛЕКСИЯ

API-тестирование оказалось гораздо быстрее и стабильнее UI-тестов. В отличие от UI, где нужно ждать рендеринга страницы, загрузки скриптов и обходить защиты вроде CAPTCHA, API-запросы выполняются за миллисекунды и дают точный HTTP-статус. Интеграция тестов в CI с помощью GitHub Actions показала важность принципа Shift-Left: разработчик получает обратную связь о сломанном функционале сразу после пуша кода, ещё до того, как изменения попадут на рабочий сервер.

6 ВЫВОДЫ

В ходе лабораторной работы изучены методы ручного и автоматизированного API-тестирования. Разработан набор из 10 автотестов с помощью библиотеки `requests`, покрывающий основные проверки безопасности, контрактов и бизнес-логики. Тесты успешно интегрированы в CI-пайплайн GitHub Actions, что обеспечивает постоянный контроль качества при внесении изменений в код.